Lab 15 — Commented answers

Each answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, an example you can verify at the terminal.

Question 1 — `hostPath` for a database

Answer. It "works" because k3s has a single node: `/data/pg` is always the same directory, on the same machine. On three nodes, the Pod recreated after a failure lands on another node, finds an **empty** `/data/pg`, and PostgreSQL initialises a fresh database: the data is "lost" (actually left behind on the dead node). The PVC → PV → StorageClass chain brings a volume **independent of the node** (a network disk attached wherever the Pod runs), a life cycle managed by the cluster, and a portable request: the PVC does not say *where*, only *how much* and *how*.

Why. `hostPath` ties the Pod to a machine; a PVC ties the Pod to a volume, which the platform knows how to move.

Nuance. With `local-path`, the PV receives a **node affinity**: the Pod will never be scheduled elsewhere — the same limit as `hostPath`, but declared and visible (`Pending` with `FailedScheduling` instead of an empty database).

Example.

```
kubectl get pv -o
jsonpath='{.items[0].spec.nodeAffinity.required.nodeSelectorTerms[0].matchExpressions[0].values
[0]}' ; echo   # wsl-ubuntu
```

Question 2 — `Pending`, and that is normal

Answer. It is not an error: the `local-path` StorageClass has `volumeBindingMode: WaitForFirstConsumer`. The volume is created — and the PVC bound — only when a **Pod** referencing the PVC is scheduled, so that the volume can be created on the chosen node. As soon as a Pod uses it, the PVC turns `Bound` within seconds.

Why. Creating the volume before knowing where the Pod will run would force the scheduler to follow the volume; the reverse is healthier.

Nuance. With `volumeBindingMode: Immediate` (some network classes), the PVC is bound right away. `kubectl describe pvc` shows `waiting for first consumer to be created before binding`: the sentence to recognise.

Example.

```
kubectl describe pvc data | grep -i 'waiting for first consumer'
```

Question 3 — The fate of the data

Answer. (a) Pod deletion: **nothing** happens to the data; the PVC stays `Bound`, a new Pod remounts it. (b) PVC deletion: the PV, under policy `Delete`, is **deleted with its data**. (c) Namespace deletion: every PVC in the namespace is deleted, hence (b) for each. (d) `replicas: 0` on a StatefulSet: the Pods disappear, the `data-db-0` PVCs **stay**; going back to `replicas: 1` reattaches them.

Why. The PVC is the object that "holds" the volume; as long as it exists, the data exists; its deletion triggers the reclaim policy.

Nuance. `reclaimPolicy: Retain` would keep the PV (in `Released` state) after (b) and (c), to be cleaned up or reattached by hand — safer, less tidy.

Example.

```
kubectl scale statefulset db --replicas=0 && kubectl get pvc     # data-db-0 still Bound
```

Question 4 — Deployment versus StatefulSet for a database

Answer. Deployment: Pod `db-7d9f8b6c5-x4k2p`, renamed at every recreation; a hand-written, single `pgdata` PVC; an ordinary ClusterIP Service; at `replicas: 3`, the three Pods try to mount the **same** `ReadWriteOnce` PVC — two stay `Pending` (`Multi-Attach error`) or, on a single node, three PostgreSQL instances fight over the same directory. StatefulSet: Pod `db-0`, name kept; PVC `data-db-0` generated by `volumeClaimTemplates`; headless Service; at `replicas: 3`, three Pods `db-0`, `db-1`, `db-2` with **three** distinct PVCs, started in order — three independent databases.

Why. The Deployment treats its Pods as interchangeable; the StatefulSet gives them an identity and storage of their own.

Nuance. For a single instance that will never be scaled, a Deployment with a PVC and `strategy: Recreate` (no rolling update, so as never to have two Pods on the disk) is acceptable; the StatefulSet remains more explicit.

Example.

```
kubectl get pods,pvc -l app=db    # db-0 / data-db-0
```

Question 5 — The headless Service

Answer. A headless Service has `clusterIP: None`: no virtual IP, no distribution by kube-proxy. The DNS name `db.lab15.svc.cluster.local` returns the Pods' IPs directly, and each StatefulSet Pod additionally gets its own name: `db-0.db.lab15.svc.cluster.local`. The `api` Service of lab 13 hid the Pods behind a stable IP; the headless one **exposes** them individually. A StatefulSet requires one (`serviceName`) because it is what creates those per-Pod names — essential for a replica to reach `db-0` precisely.

Why. A database is not a set of interchangeable Pods: the client must sometimes choose which one (primary, replica).

Nuance. A second **regular** Service `db-rw` is often added with a selector targeting only the primary (a label set by the operator): clients write to a stable name, the operator moves the label on failover.

Example.

```
kubectl exec client -- getent hosts db db-0.db     # same IP, two names
```

Question 6 — `lost+found`

Answer. The volume is formatted as `ext4` by the driver, and `ext4` creates a `lost+found` directory at its root: for `initdb`, the directory "is not empty". The kubelet (through the storage driver) put it there. The `PGDATA=/var/lib/postgresql/data/pgdata` variable makes the database initialise in an empty **subdirectory** of the volume, as the official image recommends.

Why. `initdb` refuses any non-empty directory so as never to overwrite an existing database by mistake.

Nuance. With `local-path`, the volume is a plain directory with no `lost+found`: the problem only appears on a real disk. Putting `PGDATA` in a subdirectory from the start avoids the surprise at migration time.

Example.

```
kubectl exec db-0 -- ls /var/lib/postgresql/data     # pgdata (and lost+found on a formatted disk)
```

Question 7 — `local-path` on two nodes

Answer. The volume is created **on the node where the first Pod is scheduled** (hence `WaitForFirstConsumer`), in `/var/lib/rancher/k3s/storage/`, and the PV receives a `nodeAffinity` to that node. If that node goes down, the recreated Pod can only be scheduled there: it stays `Pending` with `FailedScheduling … node(s) had volume node affinity conflict` until the node returns. On EKS, the `gp3` StorageClass (EBS CSI driver) creates a network disk attachable to any node in the zone; the PVC **does not change** (except `storageClassName` if the default class differs).

Why. `local-path` is meant for a workstation or the edge; it honestly exposes its limit in the PV.

Nuance. Even EBS has a limit: the disk is tied to an **availability zone**; a multi-zone cluster uses `WaitForFirstConsumer` for the same reason.

Example.

```
kubectl get pv -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeAffinity.required.nodeSelectorTerms[0].matchExpressions[0].values[0]
```

Question 8 — Three databases, not one replicated database

Answer. They deployed **three independent PostgreSQL instances**: `db-0`, `db-1`, `db-2`, each with its PVC and its empty database. Kubernetes knows nothing about PostgreSQL replication (primary configuration, *streaming replication*, promoting a replica): it only manages Pods and volumes. An **operator** is required (CloudNativePG, Zalando Postgres Operator, Crunchy): it creates the primary, configures the replicas, sets labels for the read/write Services, monitors, fails over, and backs up.

Why. A database's high availability is an **application** problem; the StatefulSet only provides the building blocks (names, volumes, order).

Nuance. For systems designed for clusters (Cassandra, etcd, Elasticsearch), the StatefulSet is sometimes enough because each node discovers the others through the `db-N` names; PostgreSQL

is not one of them.

Example.

```
kubectl exec db-1 -- psql -U postgres -c '\dt'     # Did not find any relations: empty database
```

Question 9 — Deleting the StatefulSet

Answer. `get pvc` shows `data-db-0` still `Bound` : deleting the StatefulSet does not touch the PVCs. After `apply` , `db-0` is recreated, finds `data-db-0` by name, and the `SELECT` returns `2` . It is intended: a StatefulSet deleted by mistake (or replaced by a new version) must never take the data with it. To make it disappear: `kubectl delete pvc data-db-0` (policy `Delete`), or delete the namespace.

Why. The PVC belongs to the namespace, not to the StatefulSet; only `persistentVolumeClaimRetentionPolicy` (configurable) changes that behaviour.

Nuance. Orphaned PVCs cost storage and get forgotten: run `kubectl get pvc` regularly on shared namespaces.

Example.

```
kubectl delete statefulset db && kubectl get pvc     # data-db-0 Bound
```

Question 10 — Access modes

Answer. `ReadWriteOnce` : a block disk (EBS, Azure Disk, `local-path`) mounted read-write by **a single node** — a database. `ReadOnlyMany` : a read-only volume across several nodes — reference data, a model. `ReadWriteMany` : a shared filesystem (NFS, EFS, Azure Files, CephFS) — uploads read by several front-end replicas. A block disk cannot be `ReadWriteMany` because two kernels writing to the same `ext4` filesystem without coordination corrupt it. Two Pods on two nodes with the same `ReadWriteOnce` PVC: the second stays `ContainerCreating` with `Multi-Attach error for volume` .

Why. The mode describes what the underlying storage can guarantee, not a wish.

Nuance. `ReadWriteOncePod` (recent) restricts even to a single Pod on the node: useful for a database.

Example.

```
kubectl get pvc -o custom-columns=NAME:.metadata.name,MODES:.spec.accessModes
```

Question 11 — `rsync` of the node directory

Answer. Same problem as in lab 06: a file copy while PostgreSQL writes yields an inconsistent state, sometimes beyond repair — and here, on top of that, the node path is an implementation detail of `local-path` . In a cluster: (1) the **logical backup**, `kubectl exec db-0 -- pg_dump -U postgres > dump.sql` , often in a `CronJob` that pushes to object storage; (2) the **volume snapshot** (`VolumeSnapshot`) if the CSI driver allows it, block-level consistent, combined with `pg_start_backup` or a stop. An operator automates: base backups + continuous WAL archiving to S3, and point-in-time restore.

Why. The volume belongs to the platform; the backup must go through the application or the storage API, not through the node.

Nuance. Testing the restore in a throwaway namespace is part of the procedure, as in lab 06.

Example.

```
kubectl exec db-0 -- pg_dump -U postgres | gzip > db-$(date +%F).sql.gz
```

Question 12 — Rolling update of a StatefulSet

Answer. A StatefulSet updates its Pods **one at a time**, in **descending** order (`db-2`, then `db-1`, then `db-0`), waiting for each to be `Ready` before the next; it has no `maxSurge` (never an extra Pod: there would be no PVC for it) and destroys the Pod before recreating it. A Deployment creates the new Pod first then removes the old one, several in parallel. For a database it is necessary: two instances on the same volume are forbidden, and the reverse order updates the replicas before the primary.

Why. Identity and per-Pod volume require that a name be carried by only one Pod at a time.

Nuance. `updateStrategy.rollingUpdate.partition` allows updating only the Pods with index ≥ N: a *canary* on `db-2` before the rest.

Example.

```
kubectl get statefulset db -o jsonpath='{.spec.updateStrategy}' ; echo
```

Question 13 — Managed database or database in the cluster

Answer. Criteria: (1) **skills and operating load** — does the team want to manage backups, upgrades and replication, or pay the provider to do it; (2) **availability requirement** — a managed service offers multi-zone failover with one checkbox; (3) **constraints** — data that must stay on premises, cost, latency, many test environments (where a disposable StatefulSet is unbeatable). What the application sees: managed database → an `ExternalName` Service (or just a URL in a ConfigMap) and a Secret with the credentials; database in the cluster → a Service to `db-0`/the operator's primary, and the same Secret. The API's manifests are **identical**: `SPRING_DATASOURCE_URL` and a Secret, that is all.

Why. The API only knows a name and a password; where the database lives is a platform decision.

Nuance. Many teams do both: an operator in the cluster for dev and staging, a managed service in production.

Example.

```
kind: Service
metadata: { name: db }
spec: { type: ExternalName, externalName: app-prod.abc123.eu-west-1.rds.amazonaws.com }
```